

Kubernetes Core Concepts

20-slide PDF carousel for beginner labs, troubleshooting, and interviews



```
kubectl get nodes  
kubectl get pods -A  
kubectl describe pod <name>  
kubectl logs <pod> --tail=50
```

Goal: understand what each object does, how traffic moves, and how Kubernetes heals problems.

Cluster pre-check • Pods • Deployments • Services • Ingress • HPA

How the topics connect

2/20



- First check the cluster and kubectl context.
- Pods run containers, but Deployments manage them safely.
- ReplicaSets keep the desired pod count alive.
- Services give stable network access to changing pods.
- Ingress, LoadBalancer, and HPA make the app production-like.

Your mental model

YAML = desired state
Controller = keeps desired state
Service = stable address
Probe = health signal
HPA = auto scale signal

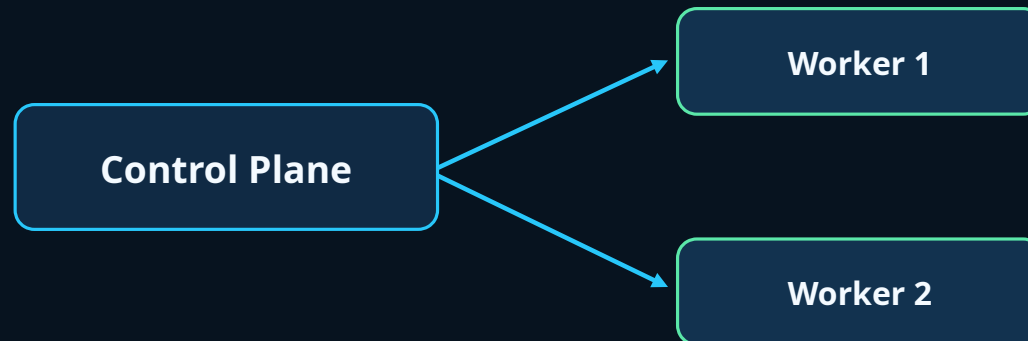
Cluster pre-check: before you deploy

3/20

Check first

- Correct kubeconfig/context?
- All nodes Ready?
- System pods healthy?
- Metrics Server installed for CPU/HPA?
- Permission to create resources?

```
kubectl config current-context  
kubectl cluster-info  
kubectl get nodes -o wide  
kubectl get pods -A  
kubectl auth can-i create deploy  
kubectl top nodes
```

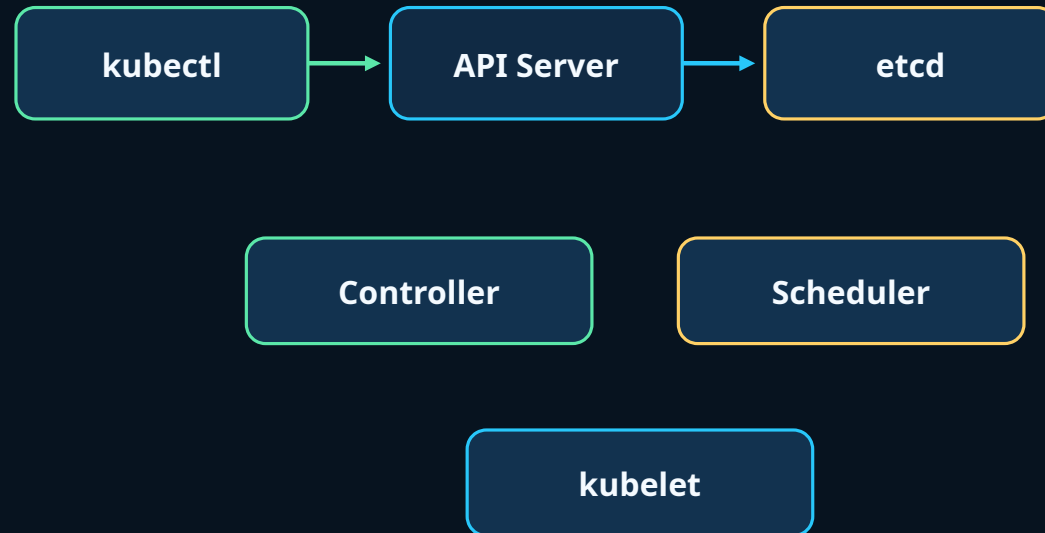


If pre-check fails, fix the cluster first. Do not blame YAML yet.

kubectl: your remote control

4/20

API flow



- kubectl sends requests to the API server.
- API server stores desired state in etcd.
- Controllers compare desired vs current state.
- Scheduler chooses where pods run.
- kubelet starts containers on the node.

```
kubectl get deploy
kubectl describe pod nginx
kubectl logs nginx --tail=50
kubectl exec -it nginx -- sh
kubectl apply -f app.yaml
```

kubectl does not run the container; it talks to the Kubernetes API.

Pods: the smallest running unit

5/20

Pod basics

- A Pod wraps one or more containers.
- Containers in one Pod share network and storage.
- Pods are temporary and can be recreated.
- For production, manage pods through Deployments.

POD

nginx container

sidecar container

shared IP

shared volume

```
kubectl run nginx --image=nginx
kubectl get pods -o wide
kubectl describe pod nginx
kubectl delete pod nginx
```

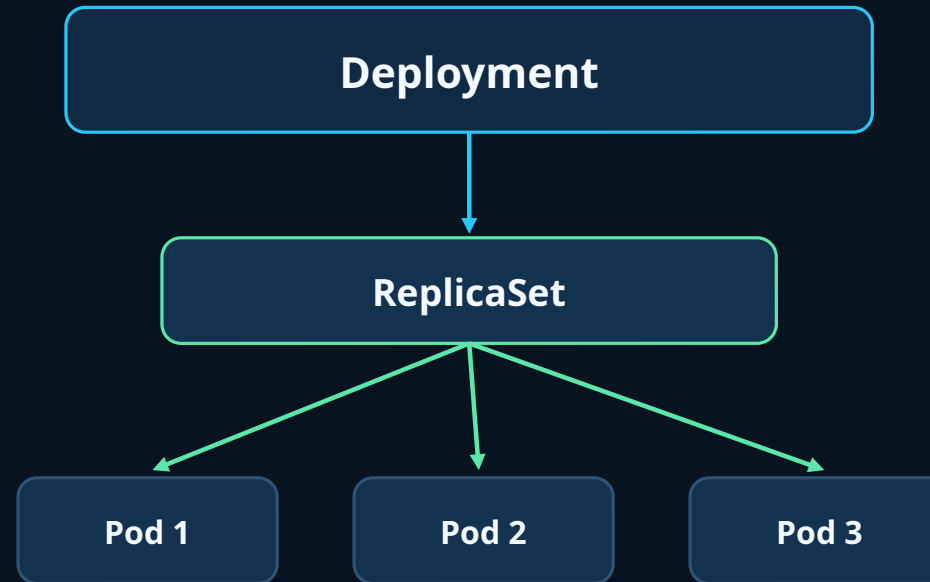
Pod deleted? Deployment/ReplicaSet can create a new one.

Deployments: safe app management

6/20

Why use it?

- Deployment manages ReplicaSets and Pods.
- It supports rolling updates and rollbacks.
- It recreates pods when image/config changes.
- Most stateless apps are deployed with Deployment YAML.



```
kubectl create deployment web --image=nginx  
kubectl get deploy,rs,pods  
kubectl set image deploy/web nginx=nginx:1.27
```

Deployment is the object you usually change, not individual pods.

ReplicaSets: desired pod count guard

7/20

Selector logic

- ReplicaSet keeps matching pods alive.
- It uses labels and selectors to find pods.
- Deployment creates/replaces ReplicaSets during updates.
- Usually inspect ReplicaSets, do not create directly.



```
kubectl get rs
kubectl describe rs <name>
kubectl get pods --show-labels
```

Wrong labels = Service/ReplicaSet cannot find the Pod.

Scaling and self-healing

8/20

Controller action

desired replicas: 4



running pods: 2

Controller creates 2 more pods

Pod 1

Pod 2

Pod 3

Pod 4

- Scale changes desired replica count.
- If a pod dies, ReplicaSet creates another.
- If a node fails, pods can be rescheduled.
- Self-healing follows desired state.

```
kubectl scale deploy/web --replicas=4
kubectl get pods -w
kubectl delete pod <pod-name>
kubectl get rs
```

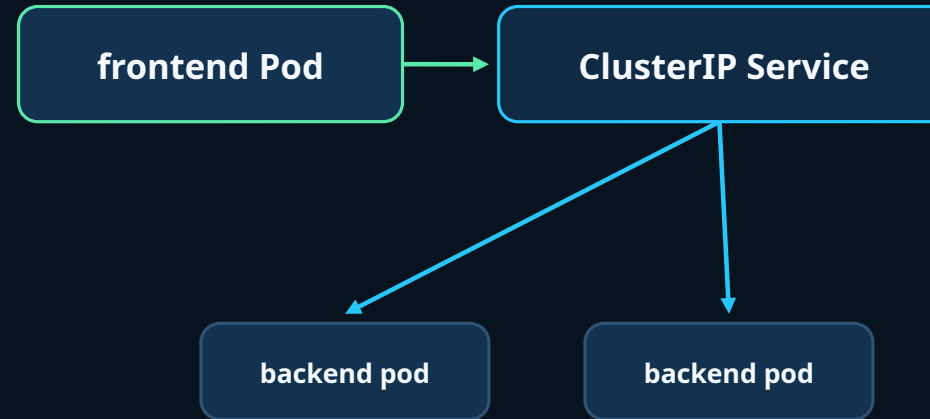
Watch mode (-w) is best for seeing self-healing live.

Services: ClusterIP

9/20

Internal access

- Default Service type for internal communication.
- Gives a stable virtual IP inside the cluster.
- Routes traffic to pods selected by labels.
- Good for frontend-to-backend communication.



```
kubectl expose deploy backend --port=80
kubectl get svc
kubectl describe svc backend
curl http://backend.default.svc.cluster.local
```

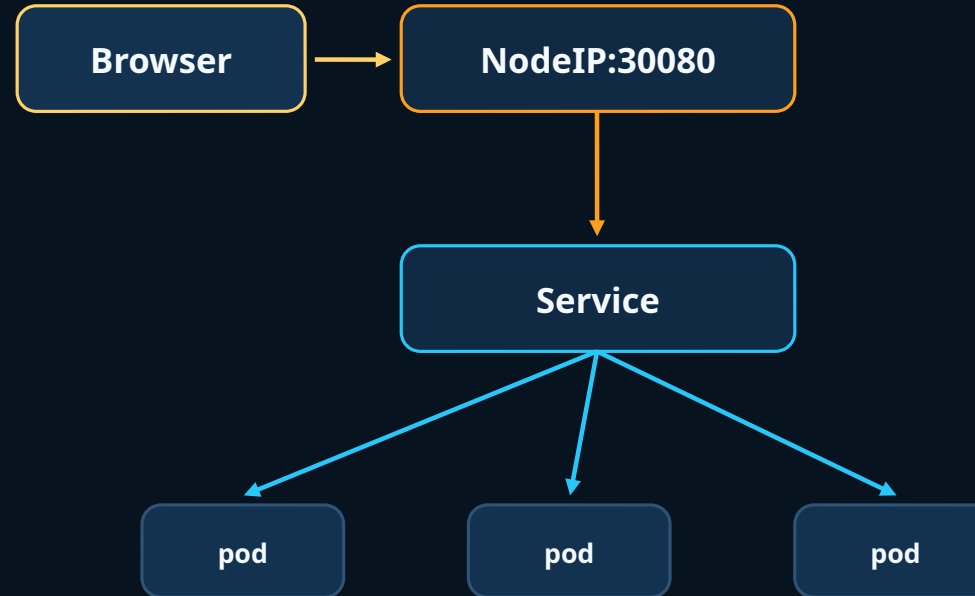
ClusterIP is not normally accessible from your laptop browser.

Services: NodePort

10/20

Lab access

- Exposes app on every node IP at a high port.
- Typical range is 30000-32767.
- Useful for local labs and quick browser testing.
- Not usually the final production entry point.



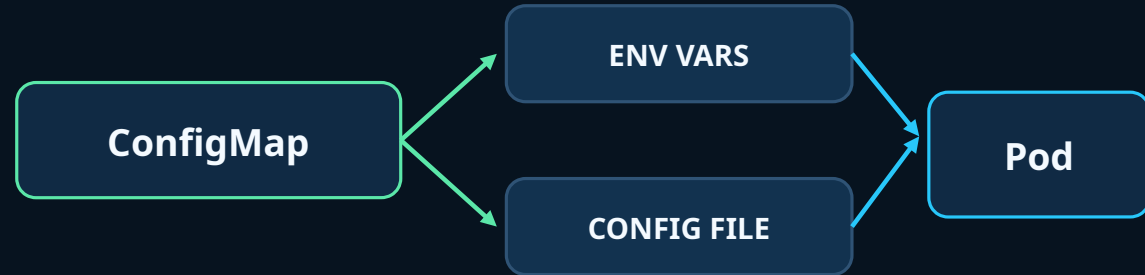
```
kubectl expose deploy web --type=NodePort --port=80
kubectl get svc web
curl http://<node-ip>:<node-port>
```

If browser fails: check node IP, firewall, service port, and endpoints.

ConfigMaps: non-sensitive configuration

11/20

Runtime config



- Use for app mode, feature flags, URLs, files.
- Keep same image but change runtime behavior.
- Mount as environment variables or files.
- Do not store passwords here.

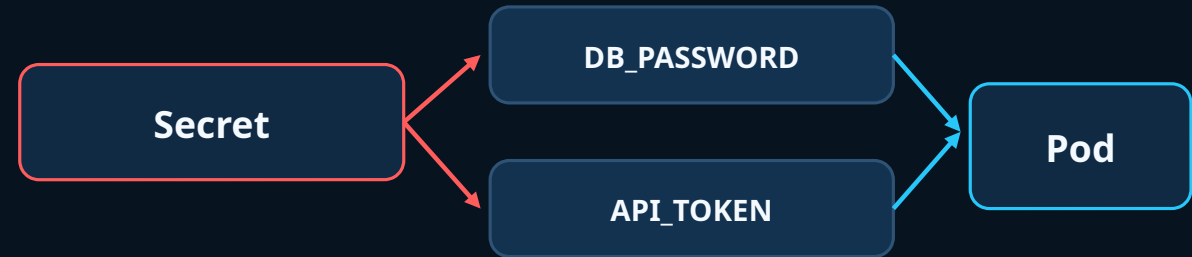
```
kubectl create configmap app-config --from-literal=APP_MODE=dev  
kubectl get cm app-config -o yaml
```

ConfigMap = non-secret settings outside the image.

Secrets: sensitive values

12/20

Sensitive data



- Use for passwords, API keys, tokens, certificates.
- Mount as environment variables or files.
- Base64 is encoding, not encryption by itself.
- In production, restrict RBAC and enable encryption at rest.

```
kubectl create secret generic db-secret --from-literal=DB_PASSWORD=StrongPass123  
kubectl get secret db-secret -o yaml
```

Never commit real passwords to Git.

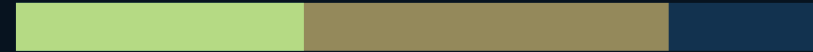
CPU and Memory Requests and Limits

13/20

Scheduling and safety

- Request = minimum resource used for scheduling.
- Limit = maximum resource allowed for the container.
- CPU over limit is throttled.
- Memory over limit can cause OOMKilled.

CPU



request 100m -> limit 500m

Memory



request 128Mi -> limit 256Mi

```
resources:  
requests: { cpu: "100m", memory: "128Mi" }  
limits: { cpu: "500m", memory: "256Mi" }
```

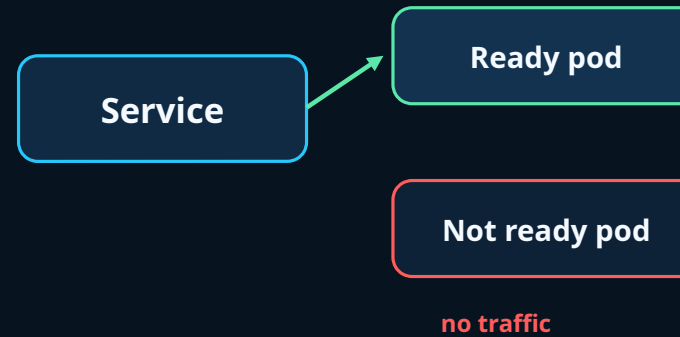
HPA CPU percentage needs a CPU request to calculate properly.

Liveness and Readiness Probes

14/20

Health checks

- Readiness: can this pod receive traffic now?
- Liveness: is this app stuck and needs restart?
- Readiness failure removes pod from Service endpoints.
- Liveness failure restarts the container.



kubelet checks /health

```
readinessProbe: httpGet: { path: /, port: 80 }  
livenessProbe: httpGet: { path: /, port: 80 }
```

Liveness failure = restart. Readiness failure = stop sending traffic.

Rolling Updates and Rollbacks

15/20

Safe releases

- Rolling update replaces old pods gradually.
- maxSurge allows extra temporary pods.
- maxUnavailable limits downtime during update.
- Rollback returns to a previous ReplicaSet revision.



```
kubectl set image deploy/web nginx=nginx:1.27
kubectl rollout status deploy/web
kubectl rollout history deploy/web
kubectl rollout undo deploy/web
```

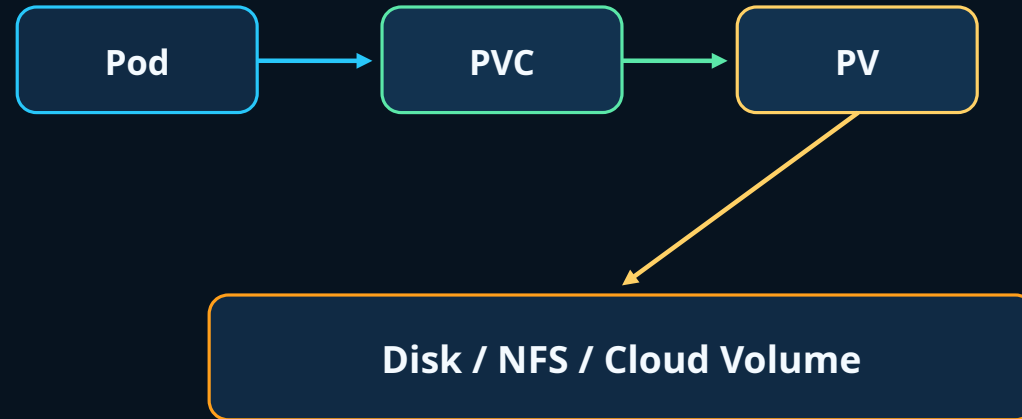
Rollback is fast because old ReplicaSet history is kept.

Persistent Volumes and PersistentVolumeClaims

16/20

Storage model

- Pod storage is normally temporary.
- PVC is a request for storage by the app.
- PV is the actual storage resource.
- StorageClass can dynamically create PVs in cloud clusters.



```
kubectl get pv,pvc
kubectl describe pvc <claim-name>
kubectl exec -it <pod> -- df -h
```

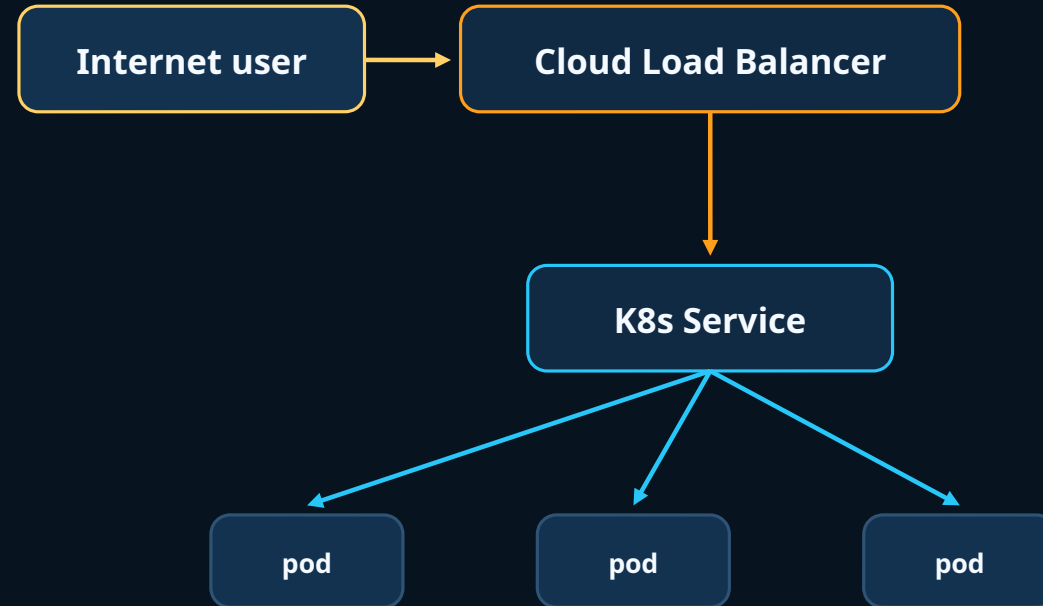
PVC is like “I need 1Gi storage.” PV satisfies it.

LoadBalancer Services

17/20

External access

- Exposes a Service through a cloud load balancer.
- Cloud provider gives an external IP or DNS name.
- Common on AKS, EKS, GKE, and managed clusters.
- Each LoadBalancer Service can create separate cost.

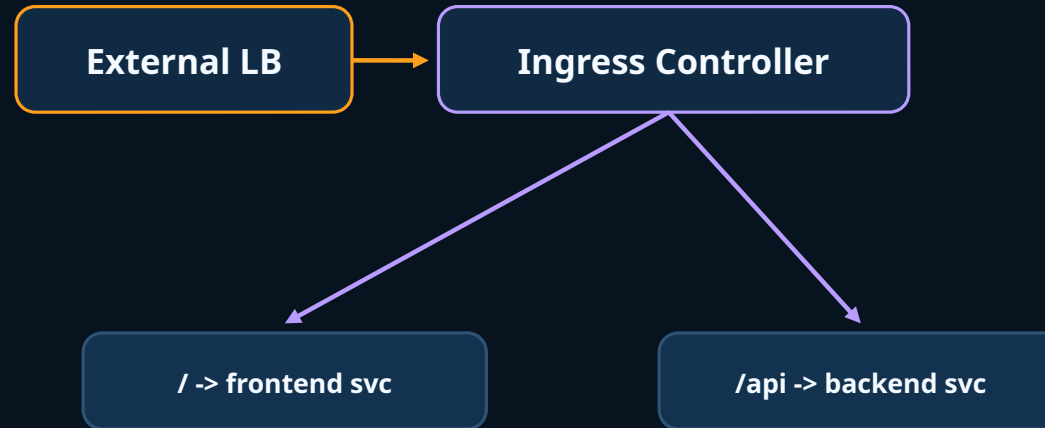


```
kubectl expose deploy web --type=LoadBalancer --port=80  
kubectl get svc web -w
```

In cloud labs, delete unused LoadBalancer services to control cost.

Smart HTTP routing

- LoadBalancer gets traffic into the cluster.
- Ingress Controller understands HTTP rules.
- One external entry can route to many services.
- Routes can use hostnames and paths.



```
kubectl get ingress
kubectl describe ingress app-ingress
kubectl get svc -n ingress-nginx
```

Without Ingress: many LoadBalancers or NodePorts. With Ingress: one entry, many routes.

HPA: Horizontal Pod Autoscaler

19/20

Auto scaling

Metrics Server

HPA

Deployment replicas

2 pods

5 pods

- HPA watches metrics such as CPU or memory.
- It compares current usage with the target.
- It updates Deployment replica count automatically.
- Requires Metrics Server or external/custom metrics.

```
kubectl autoscale deploy web --cpu-percent=60 --min=2 --max=10  
kubectl get hpa  
kubectl top pods
```

If replicas do not increase: check CPU requests, Metrics Server, and real load.

Final troubleshooting map

20/20

1. Cluster

context, nodes, system pods

2. Workload

pods, deploy, rs, events

3. Health

logs, probes, restarts

4. Config

ConfigMap, Secret, env

5. Network

Service, endpoints, ingress

6. Scale

requests, limits, HPA metrics

```
kubectl get events --sort-by=.lastTimestamp
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl get svc,endpoints,ingress
kubectl rollout status deploy/<name>
kubectl top pods
```

Best habit: always move from cluster -> workload -> pod -> network -> scale.